

Modeling and Simulation

MODULE 3

MODELING CONTINUOUS SYSTEMS

SUBTOPIC 1

CONTINUOUS SYSTEMS

OBJECTIVES

- Describe a continuous systems framework.
- Illustrate the mechanics of modeling continuous systems.
- Identify numerical techniques associated with modeling continuous systems.



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

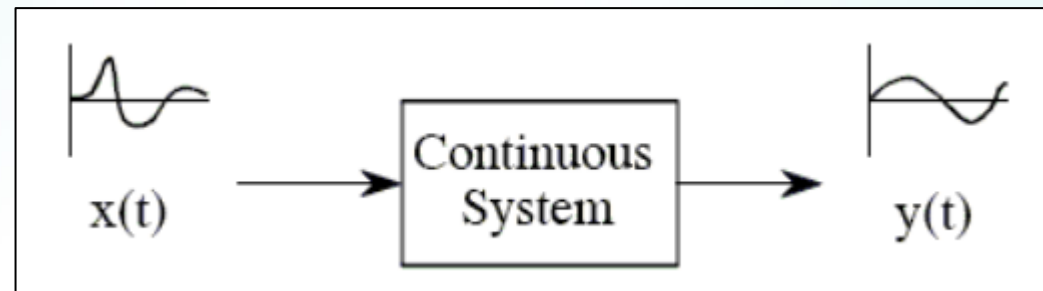
Modeling Continuous Systems

What is a continuous system?

- Continuous systems are those types of systems in which input and output signals are the same at both the ends.
- In this type of system, variable changes with time and any type of variation is not found in the input and output signal.
- In response to the input signal, a continuous system generates an output signal.

Example:

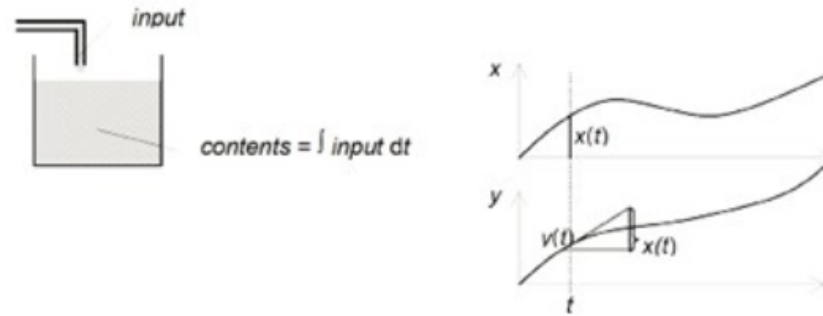
Two variables x and y and both vary with time. Continuous signals are represented within parenthesis.



Source: <https://www.javatpoint.com/continuous-systems-vs-discrete-systems>

Modeling Continuous Systems

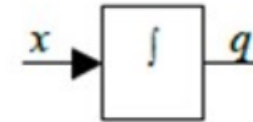
Another example:



One can imagine it as a reservoir with infinite capacity. Whatever is put into the reservoir is accumulated – with a negative input value meaning outflow. The state of the reservoir is its current contents. When we want to express this in equation form we need a variable to represent the current contents. This is our state variable q . The current input x represents the rate of current change of the contents which we express by equation

$$d q(t)/dt=x(t)$$

And the output y is equal to the current state $y(t)=q(t)$.



<https://slideplayer.com/slide/5735712/>

Modeling Continuous Systems

Application Areas

- In civil engineering for the construction of dam embankment and tunnel constructions.
- In military applications for simulation of missile trajectory, simulation of fighter aircraft training, and designing & testing of intelligent controller for underwater vehicles.
- In logistics for designing of toll plaza, passenger flow analysis at the airport terminal, and proactive flight schedule evaluation.
- In business development for product development planning, staff management planning, and market study analysis.

https://www.tutorialspoint.com/modelling_and_simulation/modelling_and_simulation_continuous_simulation.htm



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Modeling Continuous Systems – System Class

Mathematical Classes of Continuous Systems

Mathematical classes of some continuous systems

Name	Some Uses	Formula
First-order ordinary	Predator–prey models (others)	$\dot{x} = xf(x,y)$ $\dot{y} = yg(x,y)$
Second-order ordinary	Orbits Oscillators Ballistics (many, many more)	$\mathbf{F} = m\mathbf{a}$
	RLC circuits	$L\ddot{q} + R\dot{q} + \frac{1}{C}q = V(t)$
Second-order partial (not covered herein)	Water waves Sound waves Electromagnetic radiation Heat transfer Chemical diffusion Schrödinger's equation	$\nabla^2 f = \frac{1}{D} \frac{\partial f}{\partial t} - \frac{1}{v^2} \frac{\partial^2 f}{\partial t^2}$ $i\hbar \frac{\partial \psi}{\partial t} = -\frac{\hbar^2}{2m} \nabla^2 \psi + V\psi$ etc.

NOTE: a very large fraction of continuous systems are governed by second - order differential equations



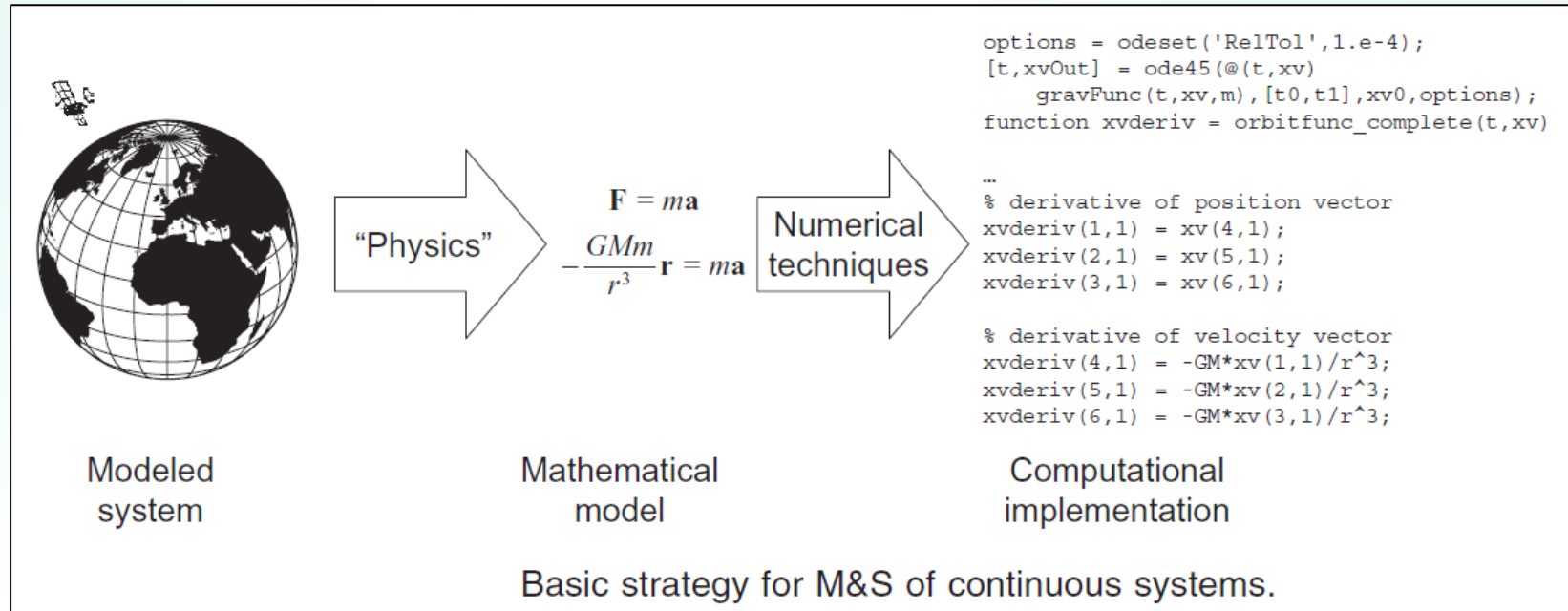
Modeling Continuous Systems – M&S Strategy

Modeling and Simulation Strategy

1. Understanding the physics of the system.

Note: **Physics** means translation of a real-world system into a set of quantitative mathematical equations.

2. Identifying the appropriate numerical implementation for solving the resulting equations.



Modeling Continuous Systems – Modeling Approach

State Variables

- Evolve in time continuously
- Continuous, meaning real or complex rather than integer valued
- Must be complete
- Must contain sufficient information to elicit all the relevant phenomena associated with the system with adequate accuracy and precision, as set forth during requirements definition.

Examples:

position, velocity, mass, rate, angle, fraction, current, voltage

State Variable Equations

- Equations dictate how the state of the system changes with respect to the independent variable
- In most M&S cases = time

Modeling Continuous Systems – Modeling Approach

Output Equations

- What quantities does the consumer of the simulation want or need?
- These issues are usually addressed during the requirements definition phase of development.

Example: Planetary orbits

The state variables at hand are simply the 3D position and velocity of the planet relative to the sun, collectively called the state vector.

$$r = \sqrt{x^2 + y^2 + z^2}$$

$$v = \sqrt{v_x^2 + v_y^2 + v_z^2}$$

$$E = -\frac{GMm}{r} + \frac{1}{2}mv^2 = -\frac{GMm}{2a}$$

where G is Newton's constant, M is the mass of the sun, and m is the mass of the planet.



Modeling Continuous Systems – Model Examples

Predator-Prey Models

- These models are intended to model two species whose populations are very closely dependent on each other, much in a way they are expected to work in the wild.

Modeling Continuous Systems – Model Examples

The predator-scenario usually produces cycles in the population numbers of both species:

- (1) Prey population increases
- (2) Predators feast on plentiful prey
- (3) Predator population increases
- (4) Copious predators overhunt the prey population
- (5) Prey population decreases
- (6) Predators starve
- (7) Predator population goes down
- (8) Prey is less vulnerable to dwindled predator population
- (9) Prey species thrives

(Repeat from step 1)

Modeling Continuous Systems – Model Examples

Usually the predator-prey model is specified mathematically by the following two equations:

$$\dot{x} = xf(x, y)$$

$$\dot{y} = yg(x, y)$$

→ referred to as Kolmogorov's predator-prey model

where x – population of prey & y – population of predator.

Note that in each equation, the change in population is proportional to the current population, such that $f(x, y)$ and $g(x, y)$ define the fractional, or per capita, growth rates in the populations (f is the derivative $\ln x$).

In the model, it is said that as the predator population, y , increases, the growth rate of the prey population of x , declines. (steps 3-5).



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Modeling Continuous Systems – Model Examples

It is further assumed that as the prey population, x , increases, the predator growth rate increases. (steps 1-3).

Lotka-Volterra Model

$$\begin{aligned}f &= b - py \\g &= rx - d\end{aligned}$$

NOTE: The model is designed such that f is linear in y with a negative slope, and g is linear in x with a positive slope.

Modeling Continuous Systems – Model Examples

After simplifying the equations using formulas by Kolmogorov's and Lotka-Volterra models:

$$C = b \ln y - py - x + d \ln x$$

This is a conservation law that holds for all $x(t)$ and $y(t)$.

The particular forms of $x(t)$ and $y(t)$ depend on the choices of b , p , r and d , and the initial conditions at time zero.



FEU ALABANG



FEU DILIMAN



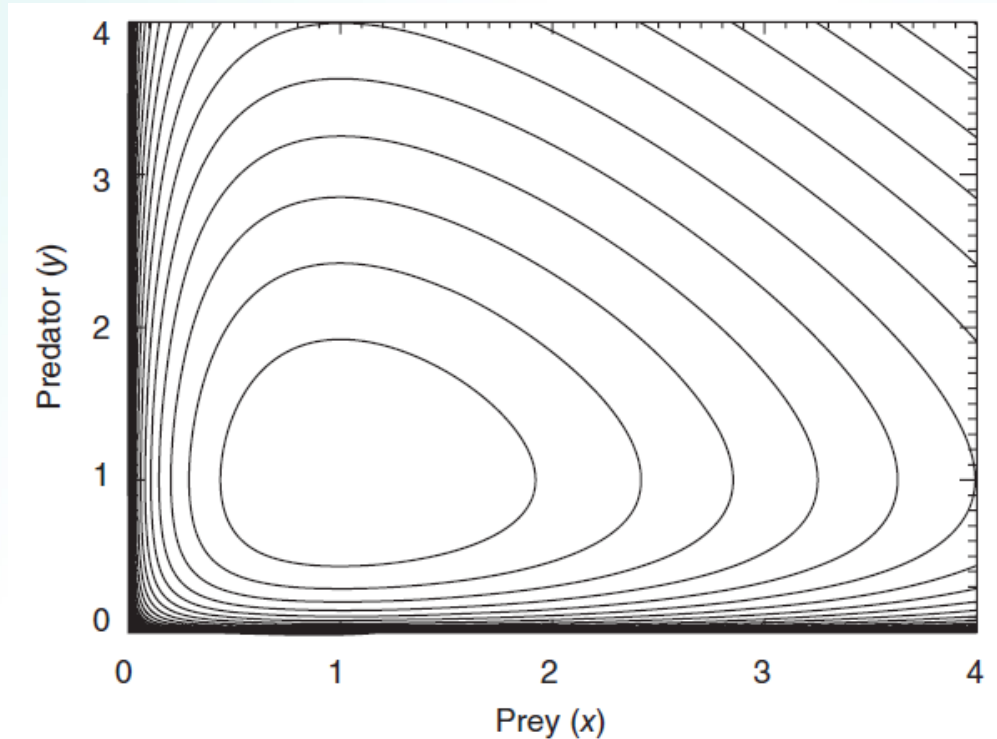
FEU TECH

Technology Driven by Innovation

Modeling Continuous Systems – Model Examples

Example:

Lotka-Volterra predator-prey model: “orbits” for different values of C , with $r = b = d = p = 1$.



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Simulating Continuous Systems

Numerical Solution Techniques – used since most systems yield equations that are not integrable.

1. Euler's Method (Tangential Method)

$$\frac{dx}{dt} \approx \frac{x(t + \Delta t) - x(t)}{\Delta t}$$
$$x(t + \Delta t) \approx x(t) + \left. \frac{dx}{dt} \right|_{x,t} \Delta t$$

$$x(t + \Delta t) \approx x(t) + f(x, t) \Delta t.$$



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Simulating Continuous Systems

2. Runge-Kutta Method

$$t_{1/2} = t_0 + \frac{\Delta t}{2};$$

$$x_{1/2} = x_0 + f(t_0, x_0) \frac{\Delta t}{2};$$

$$x_1 = x_0 + f(t_{1/2}, x_{1/2}) \Delta t.$$



Formative Assessment 4

Check Canvas for instructions



Summative Assessment 3

Check Canvas for instructions



References

Online:

<https://www.javatpoint.com/continuous-systems-vs-discrete-systems>

<https://slideplayer.com/slide/5735712/>

<https://www.tutorialspoint.com/>

E-Books and Textbooks:

- Sokolowski, John A. & Banks, Catherine M. (2010). *Modeling and Simulation Fundamentals Theoretical Underpinnings and Practical Domains*. John Wiley & Sons

MATLAB Programming Basics (Modeling and Simulation)



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Subtopic 2

MATLAB Programming:

MATLAB Programming Basics

(Input, Output, & Decision Statement)



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

OBJECTIVES

- Construct MATLAB scripts.
- Create user-defined functions that returns a single value.
- Identify and use MATLAB basic input and output commands.
- Construct MATLAB scripts with selections and looping.



FEU ALABANG



FEU DILIMAN

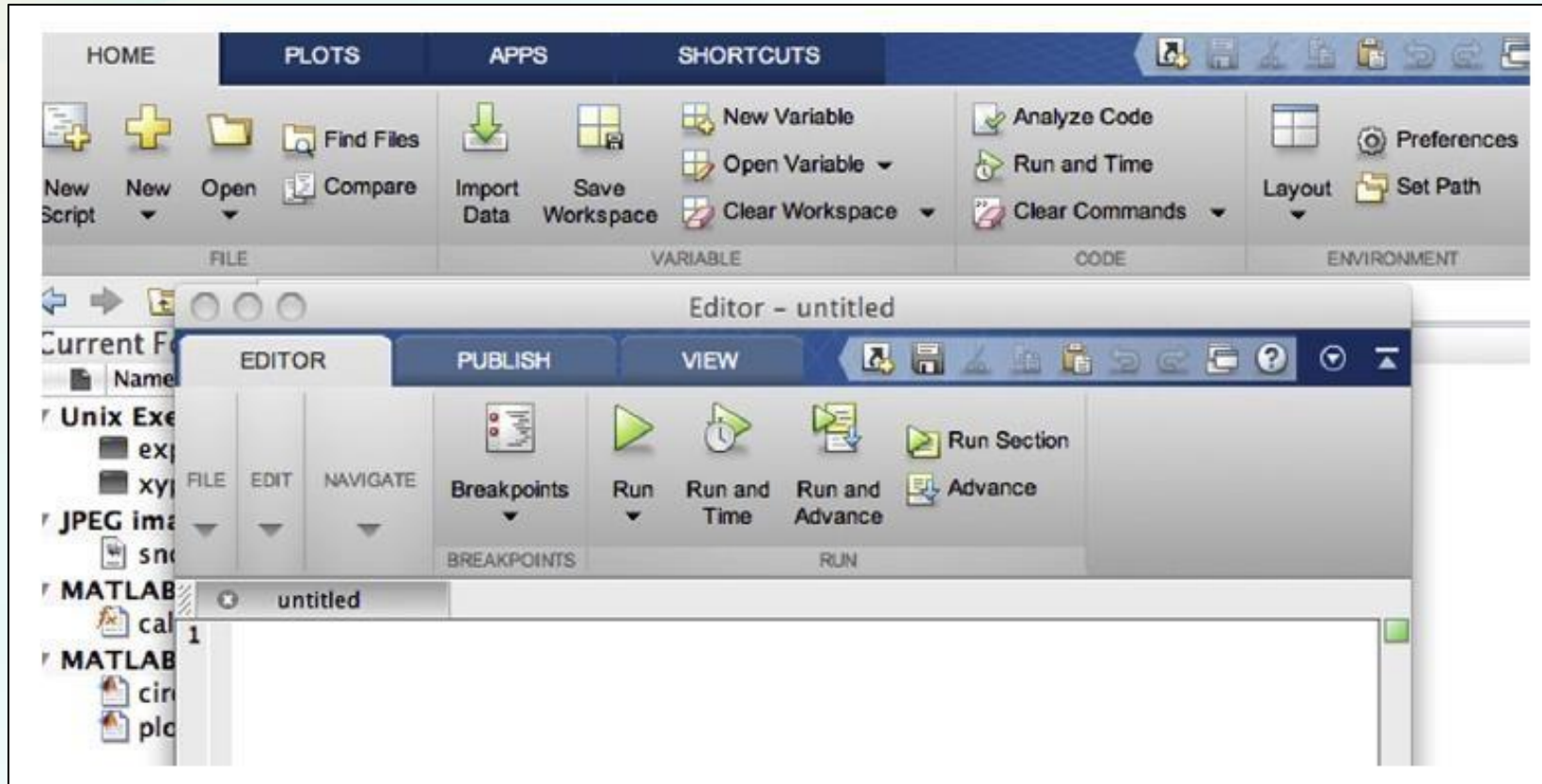


FEU TECH

Technology Driven by Innovation

Input and Output

Toolstrip and Editor



* Scripts are stored as <filename>.m

matlab.mathworks.com

HOME PLOTS APPS

New Script New Live Script New Open Upload Go to File Find Files Import Data Save Workspace New Variable Open Variable Favorites Clear Commands Simulink Layout Parallel Add-Ons

FILE VARIABLE CODE SIMULINK ENVIRONMENT

/ > MATLAB Drive >

Current Folder

Name	Git
untitled4.m	...
untitled3.m	...
untitled3.asv	...
untitled1.slx.autosave	...
untitled.slx.autosave	...
textdata.out	...
testfile.day	...
testfile.dat	...
test.mat	...
scope1.slx.autosave	...
scope1.slx	...
sample.mat	...

Workspace

Name	Value	Size	Class
a	"Stateflow"	1×1	string
ans	[20,19,20,19,16]	1×5	double
b	[7,9,11]	1×3	double
c	3	1×1	double
evec	[4,11,121]	1×3	double

matlab.mathworks.com

HOME PLOTS APPS EDITOR PUBLISH FILE VERSIONS VIEW

New Open Save Go To Find Refactor Run Section Run and Advance Run Step Stop

FILE NAVIGATE CODE SECTION RUN

/ > MATLAB Drive >

Current Folder

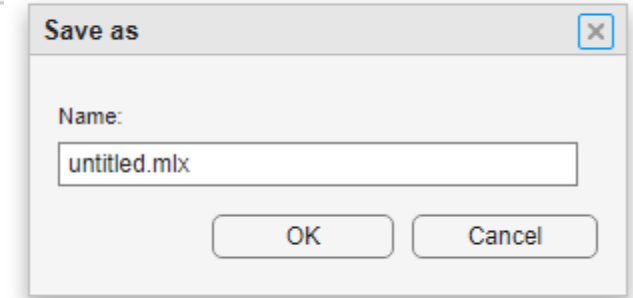
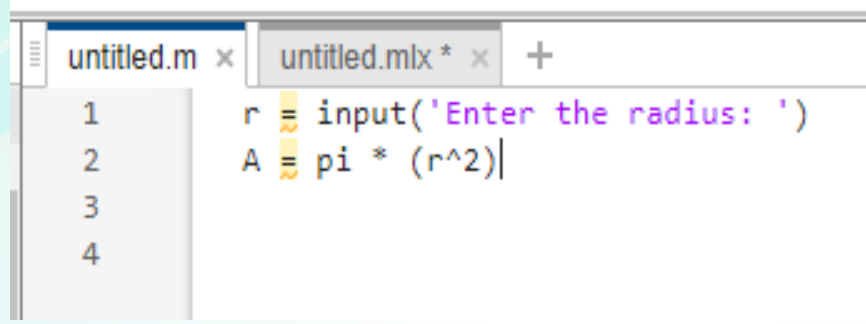
Name	Git
untitled4.m	...
untitled3.m	...
untitled3.asv	...
untitled1.slx.autosave	...
untitled.slx.autosave	...
textdata.out	...
testfile.day	...
testfile.dat	...
test.mat	...
scope1.slx.autosave	...
scope1.slx	...
sample.mat	...

Workspace

Name	Value	Size	Class
a	"Stateflow"	1×1	string
ans	[20,19,20,19,16]	1×5	double
b	[7,9,11]	1×3	double
c	3	1×1	double
evec	[4,11,121]	1×3	double
mat	3×5 double	3×5	double

untitled x 1

The M Files



MATLAB allows writing two kinds of program files:

- **Scripts** – script files are program files with **.m extension**. In these files, you write series of commands, which you want to execute together. Scripts do not accept inputs and do not return any outputs. They operate on data in the workspace.
- **Functions** – functions files are also program files with **.m extension**. Functions can accept inputs and return outputs. Internal variables are local to the function.

MATLAB Programming Basics

Creating and Running Script File

To create scripts files, you need to use a text editor.

You can open the MATLAB editor in two ways:

- Using the command prompt
- Using the IDE

If you are using the command prompt, type **edit** in the command prompt. This will open the editor. You can directly type **edit** and then the filename (with .m extension)

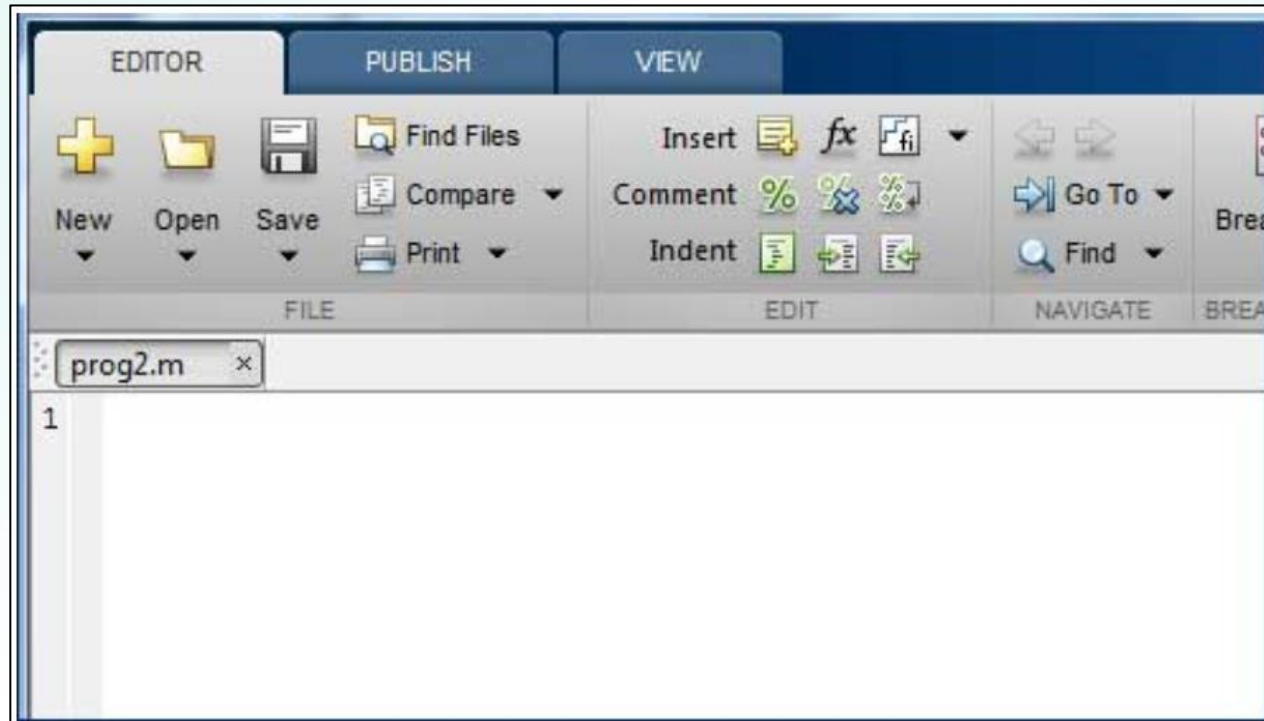
```
edit  
Or  
edit <filename>
```


MATLAB Programming Basics

Creating Directory for Saving Scripts

Example:

```
mkdir progs    % create directory progs under default directory  
chdir progs    % changing the current directory to progs  
edit prog1.m   % creating an m file named prog1.m
```



Example of Scripts

script1.m

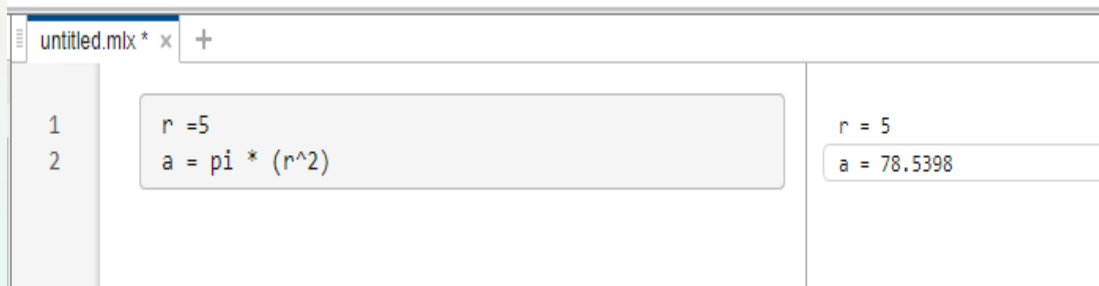
```
radius = 5  
area = pi * (radius^2)
```



```
>> type script1  
radius = 5  
area = pi * (radius^2)
```



```
>> script1  
radius =  
      5  
area =  
      78.5398
```



Figures

Area Formula

Variables

Area of Rectangle

Area = $l \times w$

l = length w = width

Area of Square

Area = a^2

a = sides of the square

Area of a Triangle

Area = $1/2 \times b \times h$

b = base h = height

Area of a Circle

Area = πr^2

r = radius of the circle



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Examples of script

```
s1 = ["Red" "Blue" "Green"]  
s2 = ["Truck" "Sky" "Tree"]  
s = s1 + s2
```

```
a = 1:2;  
b = (1:3)';  
a + b
```

```
A = [1 0; 2 4];  
B = [5 9; 2 1];  
C = A + B
```



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Examples of Using Input Commands (script)

% entering an integer

```
input("Enter an integer : ")
```

Output :

Enter an integer : 10

ans = 10

% entering a string

```
input("Enter a string : ", "s")
```

Enter a string : FEU_Tech

ans = FEU_Tech

```
x = input("Enter a number")
```

```
y = input("Enter another number")
```

```
z1 = x + y
```

```
z2 = x - y
```

```
z3 = x * y
```

```
z4 = (x + y)/2
```



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

•**Functions** –functions files are also program files with **.m extension**. Functions can accept inputs and return outputs. Internal variables are local to the function.

```
x = input("Enter a number ")
y = input("Enter another number ")
z1 = x + y
z2 = x - y
z3 = x * y
z4 = (x + y)/2
fprintf('\n Sum is. %d ',z1);
fprintf('\n Difference is. %d',z2);
fprintf('\n Product is. %d',z3);
fprintf('\n Average is. %d',z4);
```



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

disp(X) displays the value of variable X without printing the variable name. Another way to display a variable is to type its name, which displays a leading “X =” before the value.

```
A = [15 150];  
disp(A)
```

```
S = 'Hello World.';  
disp(S)
```

```
disp('Corn     Oats     Hay')
```

```
X = rand(5,3);  
disp(X)
```

```
X = '<a href =  
"https://www.mathworks.com">MathWorks  
Web Site</a>';  
disp(X)
```

fprintf to directly display the text without creating a variable. However, to terminate the display properly, you must end the text with the newline (\n) metacharacter.

```
name = 'Alice';  
age = 12;  
fprintf('%s will be %d this year.\n',name,age);
```



Input and Output Commands

Command	Purpose
disp	Displays contents of an array or string.
fscanf	Read formatted data from a file.
format	Controls screen-display format.
fprintf	Performs formatted writes to screen or file.
input	Displays prompts and waits for input.
;	Suppresses screen printing.



Input and Output Commands

The **fscanf** and **fprintf** commands behave like C scanf and printf functions. They support the following format codes.

Format Code	Purpose
%s	Format as a string.
%d	Format as an integer.
%f	Format as a floating point value.
%e	Format as a floating point value in scientific notation.
%g	Format in the most compact form: %f or %e.
\n	Insert a new line in the output string.
\t	Insert a tab in the output string.



MATLAB Programming Basics

Input and Output Commands

The format function has the following forms used for numeric display:

Format Function	Display up to
format short	Four decimal digits (default).
format long	16 decimal digits.
format short e	Five digits plus exponent.
format long e	16 digits plus exponents.
format bank	Two decimal digits.
format +	Positive, negative, or zero.
format rat	Rational approximation.
format compact	Suppresses some line feeds.
format loose	Resets to less compact display mode.



MATLAB Programming Basics

Examples of Using Input Commands Interactively

```
>> rad = input('Enter the radius: ')
Enter the radius: 5
rad =
    5
```

```
>> x = input('Enter the \nx coordinate: ');
Enter the
x coordinate: 4
```

If character or string input is desired, 's' must be added as a second argument to the input function:

```
>> letter = input('Enter a char: ', 's')
Enter a char: g
letter =
    g
```

```
>> mychar = input('Enter a character: ', 's')
Enter a character:
mychar =
    ''
```

* user hit ENTER key without typing any characters

```
>> mystr = input('Enter a string: ', 's')
Enter a string:      go
mystr =
      go
>> length(mystr)
ans =
      6
```

* user types four spaces before 'go'



Examples of Using Output Commands Interactively

```
>> disp('Hello')
Hello

>> disp(4^3)
64
```

With formatting:

```
>> fprintf('The value is %d, for sure!\n',4^3)
The value is 64, for sure!
```

```
>> fprintf('The int is %d and the char is %c\n', ...
    33 - 2, 'x')
The int is 31 and the char is x
```

```
>> fprintf('The int is %3d and the float is %6.2f\n', 5, 4.9)
The int is 5 and the float is 4.90
```



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Printing Vectors and Matrices Interactively

Examples:

```
>> vec = 2:5;
>> fprintf('%d\n', vec)
2
3
4
5

>> fprintf('%d', vec)
2345>>

>> fprintf('%d %d %d %d\n', vec)
2 3 4 5
```

```
>> mat = [5 9 8; 4 1 10]
mat =
     5     9     8
     4     1    10

>> fprintf('%d\n', mat)
5
4
9
1
8
10

>> fprintf('%d %d %d\n', mat)
5 4 9
1 8 10
```



MATLAB Programming Basics

Vector, Matrix and Array Commands

Command	Purpose
cat	Concatenates arrays.
find	Finds indices of nonzero elements.
length	Computes number of elements.
linspace	Creates regularly spaced vector.
logspace	Creates logarithmically spaced vector.
max	Returns largest element.
min	Returns smallest element.
prod	Product of each column.
reshape	Changes size.
size	Computes array size.
sort	Sorts each column.



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

MATLAB Programming Basics

Vector, Matrix and Array Commands

Command	Purpose
sum	Sums each column.
eye	Creates an identity matrix.
ones	Creates an array of ones.
zeros	Creates an array of zeros.
cross	Computes matrix cross products.
dot	Computes matrix dot products.
det	Computes determinant of an array.
inv	Computes inverse of a matrix.
pinv	Computes pseudoinverse of a matrix.
rank	Computes rank of a matrix.
rref	Computes reduced row echelon form.
cell	Creates cell array.

Command	Purpose
celldisp	Displays cell array.
cellplot	Displays graphical representation of cell array.
num2cell	Converts numeric array to cell array.
deal	Matches input and output lists.
iscell	Identifies cell array.

MATLAB Programming Basics

Scripts with Input and Output

Example:

circleIO.m

```
% This script calculates the area of a circle
% It prompts the user for the radius

% Prompt the user for the radius and calculate
% the area based on that radius
fprintf('Note: the units will be inches.\n')
radius = input('Please enter the radius: ');
area = pi * (radius^2);

% Print all variables in a sentence format
fprintf('For a circle with a radius of %.2f inches,\n', radius)
fprintf('the area is %.2f inches squared\n',area)
```

```
>> circleIO
```

Note: the units will be inches.

Please enter the radius: 3.9

For a circle with a radius of 3.90 inches,
the area is 47.78 inches squared



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

MATLAB Programming Basics

Plotting Commands

Command	Purpose
axis	Sets axis limits.
fplot	Intelligent plotting of functions.
grid	Displays gridlines.
plot	Generates xy plot.
print	Prints plot or saves plot to a file.
title	Puts text at top of plot.
xlabel	Adds text label to x-axis.
ylabel	Adds text label to y-axis.
axes	Creates axes objects.
close	Closes the current plot.
close all	Closes all plots.
figure	Opens a new figure window.
gtext	Enables label placement by mouse.

Command	Purpose
hold	Freezes current plot.
legend	Legend placement by mouse.
refresh	Redraws current figure window.
set	Specifies properties of objects such as axes.
subplot	Creates plots in subwindows.
text	Places string in figure.
bar	Creates bar chart.
loglog	Creates log-log plot.
polar	Creates polar plot.
semilogx	Creates semilog plot. (logarithmic abscissa).
semilogy	Creates semilog plot. (logarithmic ordinate).
stairs	Creates stairs plot.
stem	Creates stem plot.

MATLAB Programming Basics

To plot the graph of a function:

- Define **x**, by specifying the **range of values** for the variable **x**, for which the function is to be plotted
- Define the function, **$y = f(x)$**
- Call the **plot** command, as **plot(x, y)**

To add titles, labels, grid lines and scaling on the graph

- The **xlabel** and **ylabel** commands generate labels along x-axis and y-axis.
- The **title** command allows putting a title on the graph.
- The **grid on** command allows putting grid lines on the graph.
- The **axis equal** command allows generating the plot with the same scale factors and the spaces on both axes.
- The **axis square** command generates a square plot.

MATLAB Programming Basics

Customizing a Plot: Color, Line Types, Marker Types

The **LineSpec**, or line specification, can specify up to three different properties in a string, including the color, line type, and the symbol or marker used for the data points.

The possible colors are:

- b blue
- c cyan
- g green
- k black
- m magenta
- r red
- w white
- y yellow

The plot symbols, or markers, that can be used are:

- o circle
- d diamond
- h hexagram
- p pentagram
- + plus
- . point
- s square
- * star
- v down triangle
- < left triangle
- > right triangle
- ^ up triangle
- x x-mark

Line types can also be specified by the following:

- dashed
- . dash dot
- : dotted
- solid



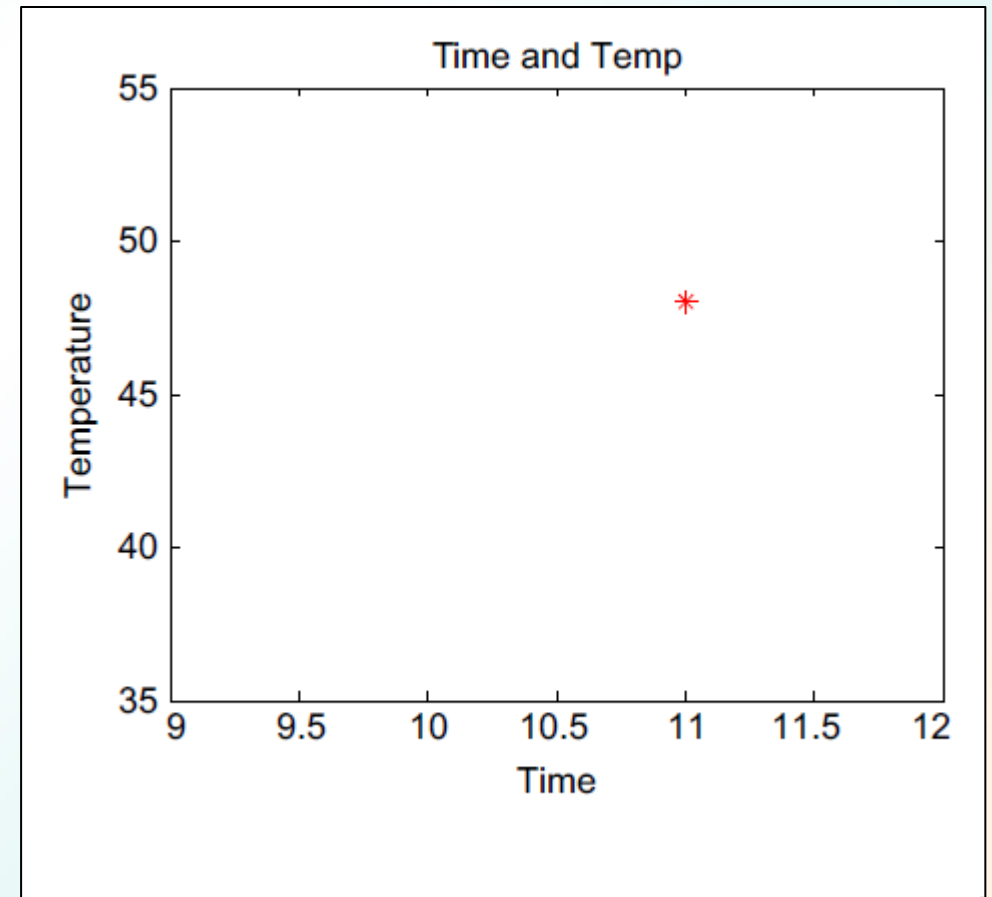
MATLAB Programming Basics

Scripts to Produce Plots

Example 1:

plotonepoint.m

```
% This is a really simple plot of just one point!  
% Create axis([x-2 x+2 y-10 y+10]) plot a red '*'  
x = 11;  
y = 48;  
plot(x,y,'r*')  
  
% Change the axes and label them axis([9 12 35 55])  
xlabel('Time')  
ylabel('Temperature')  
  
% Put a title on the plot  
title('Time and Temp')
```



FEU ALABANG



FEU DILIMAN



FEU TECH

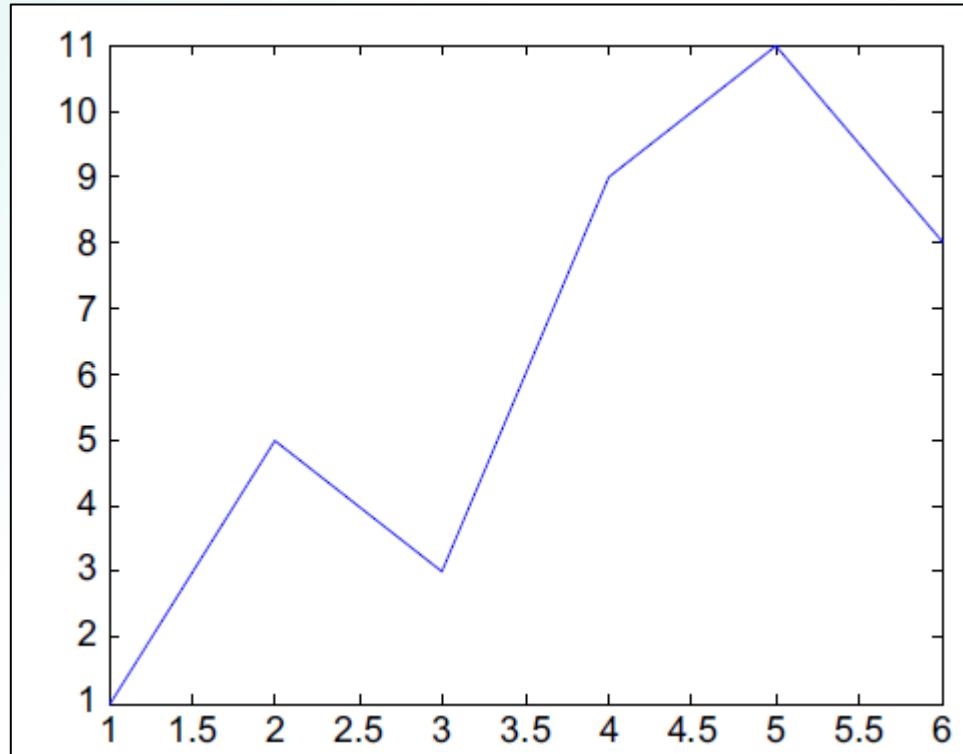
Technology Driven by Innovation

MATLAB Programming Basics

Scripts to Produce Plots

Example 2:

```
>> x = 1:6;  
>> y = [1 5 3 9 11 8];  
>> plot(x,y)
```



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

MATLAB Programming Basics

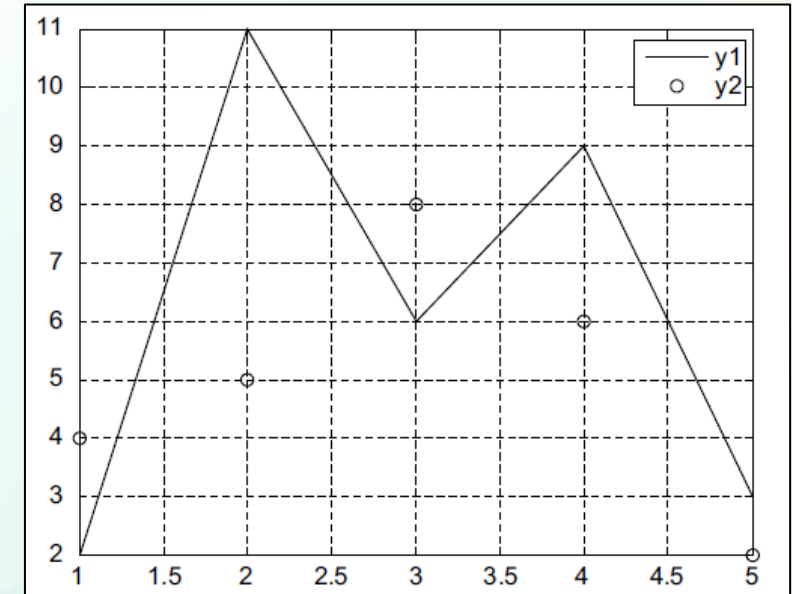
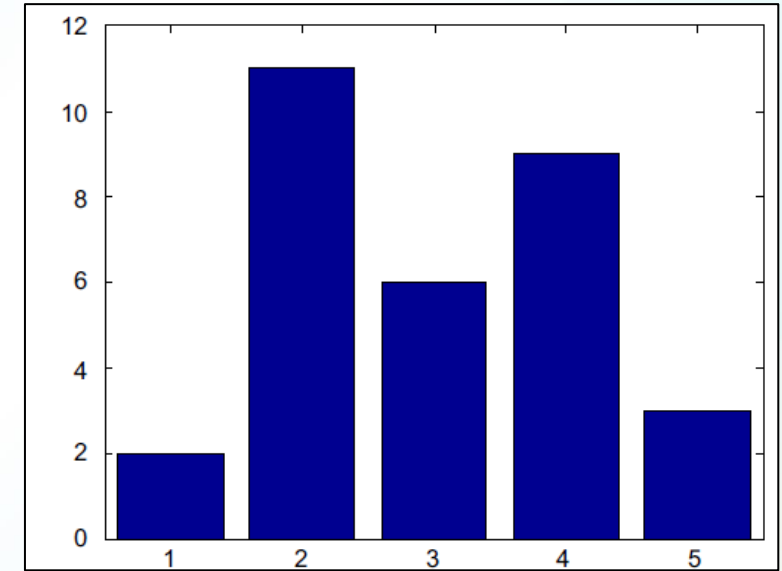
Scripts to Produce Plots

Example 3:

```
plot2figs.m

% This creates 2 different plots, in 2 different
% Figure Windows, to demonstrate some plot features

clf
x = 1:5; % Not necessary
y1 = [2 11 6 9 3];
y2 = [4 5 8 6 2];
% Put a bar chart in Figure 1
figure(1)
bar(x,y1)
% Put plots using different y values on one plot
% with a legend
figure(2)
plot(x,y1,'k')
hold on
plot(x,y2,'ko')
grid on
legend('y1','y2')
```



MATLAB Programming Basics

Scripts to Produce Plots

Example 4:

sinncos.m

```
% This script plots sin(x) and cos(x) in the same Figure Window  
% for values of x ranging from 0 to 2*pi
```

```
clf
```

```
x = 0: 2*pi/40: 2*pi;
```

```
y = sin(x);
```

```
plot(x,y,'ro')
```

```
hold on
```

```
y = cos(x);
```

```
plot(x,y,'b+') ←
```

Can be omitted but:

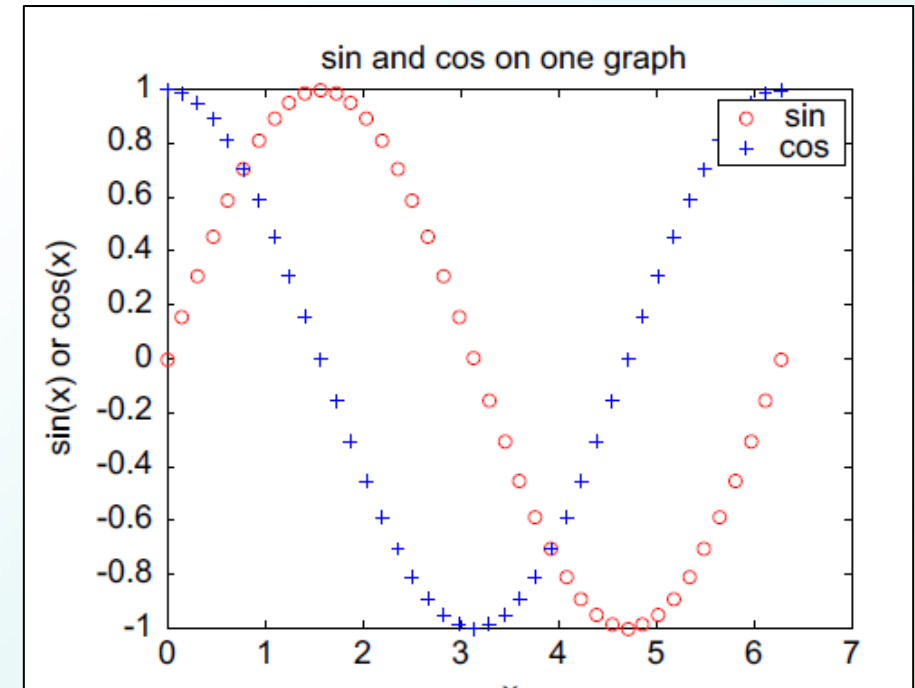
```
plot(x,sin(x),'ro',x,cos(x),'b+')
```

```
legend('sin', 'cos')
```

```
xlabel('x')
```

```
ylabel('sin(x) or cos(x)')
```

```
title('sin and cos on one graph')
```



FEU ALABANG



FEU DILIMAN



FEU TECH

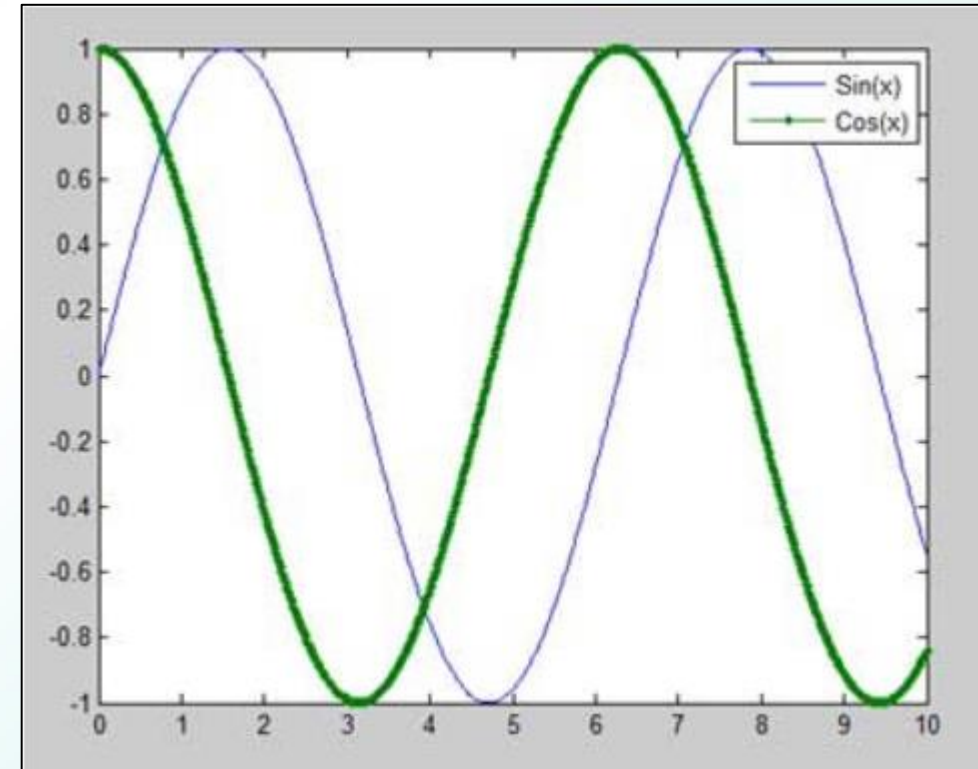
Technology Driven by Innovation

MATLAB Programming Basics

Scripts to Produce Plots

Example 5:

```
x = [0 : 0.01: 10];  
y = sin(x);  
g = cos(x);  
plot(x, y, x, g, '.-'), legend('Sin(x)', 'Cos(x)')
```



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

MATLAB Programming Basics

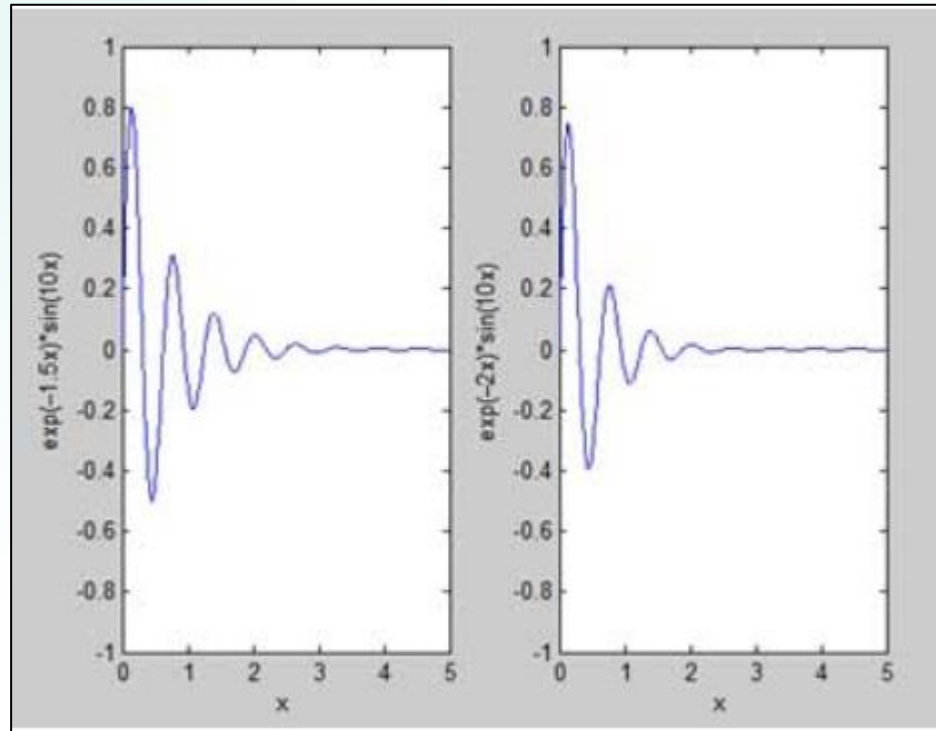
Scripts to Produce Plots

Example 6:

$$y = e^{-1.5x}\sin(10x)$$

$$y = e^{-2x}\sin(10x)$$

```
x = [0:0.01:5];  
y = exp(-1.5*x).*sin(10*x);  
subplot(1,2,1)  
plot(x,y), xlabel('x'),ylabel('exp(-1.5x)*sin(10x)'),axis([0 5 -1 1])  
y = exp(-2*x).*sin(10*x);  
subplot(1,2,2)  
plot(x,y),xlabel('x'),ylabel('exp(-2x)*sin(10x)'),axis([0 5 -1 1])
```

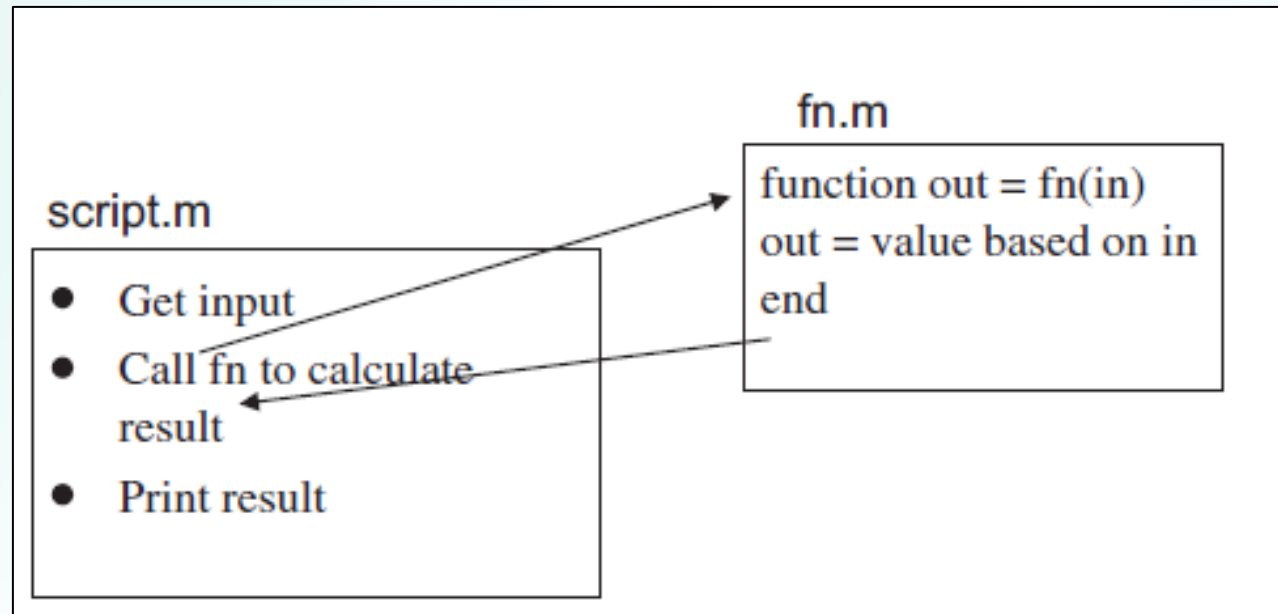


Technology Driven by Innovation

MATLAB Programming Basics

MATLAB Simple Program

General Form:



MATLAB Programming Basics

User Defined Functions

- Functions that the programmer defines, and then uses, in either the Command Window or in a script.
- Stored in M-file, which is why they are commonly called “M-file functions”.

General Form:

```
functionname.m
```

```
function outputargument = functionname(input arguments)
```

```
% Comment describing the function
```

```
Statements here; these must include putting a value in the output  
argument
```

```
end % of the function
```



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

MATLAB Programming Basics

User Defined Functions

Example 1:

calcarea.m

```
function area = calcarea(rad)
% calcarea calculates the area of a circle
% Format of call: calcarea(radius)
% Returns the area

area = pi * rad * rad;
end
```

```
>> circleCallFn
```

```
Please enter the radius: 5
```

```
For a circle with a radius of 5.00, the area is 78.54
```

circleCallFn.m

```
% This script calculates the area of a circle
% It prompts the user for the radius
radius = input('Please enter the radius: ');
% It then calls our function to calculate the
% area and then prints the result
area = calcarea(radius);
fprintf('For a circle with a radius of %.2f,',radius)
fprintf(' the area is %.2f\n',area)
```



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

MATLAB Programming Basics

User Defined Functions

Example 2:

conevol.m

```
function outarg = conevol(radius, height)
% conevol calculates the volume of a cone
% Format of call: conevol(radius, height)
% Returns the volume

outarg = (pi/3) * radius .^ 2 .* height;
end
```

```
>> conevol(4,6.1)
ans =
    102.2065
```

```
>> fprintf('The cone volume is %.2f\n', conevol(3, 5.5))
The cone volume is 51.84
```



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Selection and Looping

Selection and Looping Statements

The IF Statement

Example :

```
if condition
    action
end
```

```
if condition
    action1
else
    action2
end
```

```
a = 100;
% check the boolean condition
if a < 20
    % if condition is true then print the following
    fprintf('a is less than 20\n' );
else
    % if condition is false then print the following
    fprintf('a is not less than 20\n' );
end
fprintf('value of a is : %d\n', a);
```

```
a is not less than 20
value of a is : 100
```



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Determine if the data is negative or positive

```
.x = input("Enter a number ");  
if x < 0  
fprintf('Negative');  
else  
fprintf('Positive');  
end
```

Determine if the data is negative or positive
Odd or even number

```
x = input("Enter a number ");  
if x < 0  
fprintf('Negative');  
else  
fprintf('Positive');  
end  
y = x % 2;  
if y == 0  
fprintf('Even number');  
else  
fprintf('Even number');  
end
```



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Age problem

```
age = input("Enter a age ");
if age > 0 && age <=13
    fprintf('young \n ');
elseif age > 13 && age <20
    fprintf('teen \n ');
elseif age > 20 && age <60
    fprintf('adult \n ');
elseif age <=100
    fprintf('senior \n ');
else
    fprintf('sorry \n ');
end
fprintf("Thanks, keep safe");
```



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Selection and Looping Statements

The Switch Statement

Example :

```
switch switch_expression
case caseexp1
    action1
case caseexp2
    action2
case caseexp3
    action3
% etc: there can be many of these
otherwise
    actionn
end
```

```
grade = 'B';
switch(grade)
case 'A'
    fprintf('Excellent!\n' );
case 'B'
    fprintf('Well done\n' );
case 'C'
    fprintf('Well done\n' );
case 'D'
    fprintf('You passed\n' );
case 'F'
    fprintf('Better try again\n' );
otherwise
    fprintf('Invalid grade\n' );
end
```



```
grade = 'B';  
switch(grade)  
case 'A'  
fprintf('Excellent!\n' );  
case 'B'  
fprintf('Well done\n' );  
case 'C'  
fprintf('Well done\n' );  
case 'D'  
fprintf('You passed\n' );  
case 'F'  
fprintf('Better try again\n' );  
otherwise  
fprintf('Invalid grade\n' );  
end
```

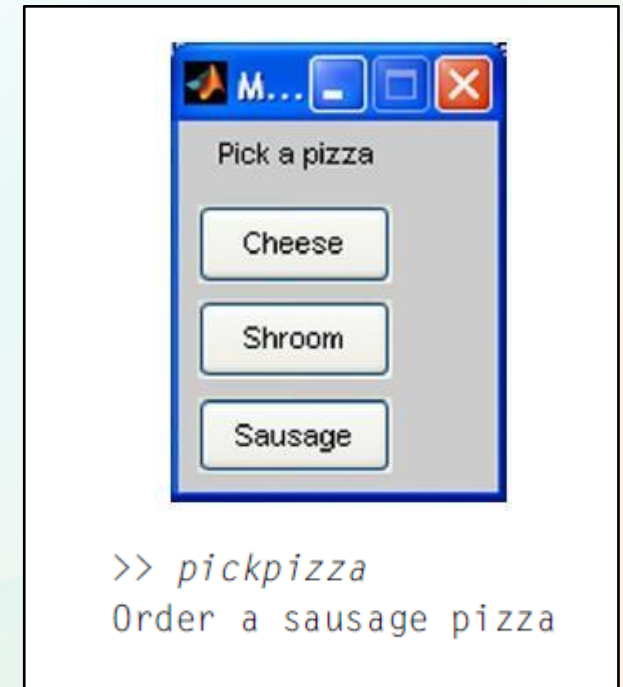
```
x = 3, y = 3;  
switch x  
case x*(x==y)  
    disp('x and y are equal');  
case x*(x>y)  
    disp('x is greater than y');  
otherwise  
    disp('x is less than y');  
end
```



The Menu Function

- MATLAB has a built-in function called menu that will display a Figure Window with pushbuttons for the options.
- The first string passed to the menu function is the heading (an instruction), and the rest are labels that appear on the pushbuttons.
- The function returns the number of the button that is pushed.

Example: %This is script as the user for a type of pizza
% and prints which type to order using a switch
pick = menu('Pick a Pizza','Cheese','Sausage','Mushroom');
switch pick
case 1
disp('Order a cheese pizza')
case 2
disp('Order a Sausage pizza')
case 3
disp('Order a Mushroom pizza')
otherwise
disp('No pizza for us today')
end



Selection and Looping Statements

The For Loop

```
for loopvar = range
    action
end
```

Example 1:

```
for a = 10:20
    fprintf('value of a: %d\n', a);
end
```

```
value of a: 10
value of a: 11
value of a: 12
value of a: 13
value of a: 14
value of a: 15
value of a: 16
value of a: 17
value of a: 18
value of a: 19
value of a: 20
```

```
for v = [1:10]
    disp(v)
end
```

=====

```
for v = [1 5 8 17]
    disp(v)
end
```

=====

```
for v = 1.0:-0.2:0.0
    disp(v)
end
```

=====

```
for v = [1:10]
    disp("Hello")
end
```



Selection and Looping Statements

The While Loop

```
while <expression>
    <statements>
end
```

Example 1:

```
a = 10;
% while loop execution
while( a < 20 )
    fprintf('value of a: %d\n', a);
    a = a + 1;
end
```

```
value of a: 10
value of a: 11
value of a: 12
value of a: 13
value of a: 14
value of a: 15
value of a: 16
value of a: 17
value of a: 18
value of a: 19
```

Example 2:

```
a = 10;
% while loop execution
while (a < 20 )
    fprintf('value of a: %d\n', a);
    a = a + 1;
    if( a > 15)
        % terminate the loop using break statement
        break;
    end
end
```

```
value of a: 10
value of a: 11
value of a: 12
value of a: 13
value of a: 14
value of a: 15
```

Example 3:

```
a = 9;
%while loop execution
while a < 20
    a = a + 1;
    if a == 15
        % skip the iteration
        continue;
    end
    fprintf('value of a: %d\n', a);
end
```

```
value of a: 10
value of a: 11
value of a: 12
value of a: 13
value of a: 14
value of a: 16
value of a: 17
value of a: 18
value of a: 19
value of a: 20
```



References

Online:

www.tutorialspoint.com

E-Books and Textbooks:

- Attaway, Stormy (2013). MATLAB ®. *A Practical Introduction to Programming and Problem Solving Third Edition*. Elsevier, Inc.



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation